

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
WEBRTC TURN + ROOM + GROUP CALL

Môn học:	Lập trình mạng
Giảng viên:	ThS. Nguyễn Thanh Quân
Nhóm thực hiện:	Đinh Nho Hoàng (23120419) Đặng Minh Ánh (23120418)

TP. Hồ Chí Minh, 2026

Tóm tắt

Hệ thống WebRTC trong đề án triển khai signaling bằng WebSocket, quản lý room theo thời gian thực và gọi nhóm theo mô hình mesh. Client hỗ trợ tạo/vào phòng, kiểm tra lỗi phòng và trùng tên, khởi tạo media khi bắt đầu gọi hoặc khi nhận offer từ peer, tự động tải ICE servers (STUN + TURN từ Metered, fallback STUN) và ghi log SEND/RECV theo thời gian thực. UI hiển thị trạng thái kết nối, cảnh báo khi P2P thất bại và thống kê loại candidate (host/srflx/relay) sau khi kết nối. Báo cáo mô tả kiến trúc, protocol signaling, luồng gọi nhóm, cấu hình ICE và kế hoạch kiểm thử bám sát code hiện tại.

Contents

Tóm tắt	1
1 Mục tiêu và phạm vi	3
2 Kiến trúc tổng thể	3
2.1 Cấu trúc thư mục	3
2.2 Thư viện và cấu hình	3
3 Thành phần server (signaling)	4
3.1 Cấu hình	4
3.2 Quản lý phòng	4
3.3 Xử lý thông điệp signaling	4
4 Thành phần client	5
4.1 Giao diện	5
4.2 Cấu hình signaling URL	5
4.3 Luồng gia nhập phòng	5
4.4 Xử lý lỗi	5
4.5 Trạng thái và cấu trúc dữ liệu	5
4.6 Xử lý WebSocket	6
4.7 Ghi log và trạng thái UI	6
4.8 Tạo PeerConnection	6
4.9 Gọi nhóm (mesh)	6
4.10 Nhận offer/answer/candidate	6
4.11 Xử lý candidate đến sớm	7
4.12 Quản lý video	7
5 Signaling protocol	7
5.1 Luồng signaling chi tiết	7
6 ICE, STUN/TURN và fallback	8
6.1 Cấu hình ICE servers	8
6.2 Fallback và trạng thái kết nối	8
6.3 Thống kê ICE candidate	8
7 Quản lý trạng thái và hangup	8
8 Đối chiếu yêu cầu đề bài	9
8.1 TURN / Kết nối qua Internet	9
8.2 Room & Group Call	9

9	Kết quả kiểm thử (bổ sung khi nộp)	9
9.1	Test cùng LAN (P2P)	9
9.2	Test khác mạng / 4G (TURN)	9
9.3	Test nhóm 3-4 người	9
10	Hạn chế và hướng phát triển	9
11	Hướng dẫn chạy nhanh	10
12	Kết luận	10

1 Mục tiêu và phạm vi

Mục tiêu của đề án là mở rộng hệ thống gọi video WebRTC với hai năng lực chính:

- Hỗ trợ kết nối qua Internet trong tình huống NAT/Firewall bằng ICE (STUN/TURN), trong đó TURN đóng vai trò relay khi P2P thất bại.
- Cho phép tạo phòng và gọi theo nhóm (mesh) nhiều người tham gia cùng lúc.

Phạm vi hệ thống:

- **Server:** Node.js + HTTPS + WebSocket (signaling).
- **Client:** HTML/CSS/JS trong trình duyệt, dùng `getUserMedia()` + `RTCPeerConnection`.
- **Đăng nhập:** không yêu cầu tài khoản/mật khẩu; dùng nickname.

2 Kiến trúc tổng thể

Hệ thống gồm 2 thành phần chính và 1 dịch vụ TURN:

- **Client (public/index.html):** giao diện lobby/room, xử lý WebRTC, tạo peer connections, hiển thị grid video và log.
- **Signaling server (server/server.js):** HTTPS + WebSocket, quản lý phòng và chuyển tiếp offer/answer/candidate.
- **TURN service (Metered):** cung cấp danh sách ICE servers (STUN/TURN) để relay khi P2P thất bại.

Mô hình kết nối dùng **mesh topology**: mỗi thành viên tạo $n-1$ kết nối `RTCPeerConnection` đến các thành viên còn lại, phù hợp quy mô nhóm nhỏ (3–4 người).

2.1 Cấu trúc thư mục

- `server/`: `server.js` (HTTPS + WebSocket signaling).
- `public/`: `index.html` (UI + client JS).
- `certs/`: `key.pem`, `cert.pem` cho HTTPS local.
- `.env.example`: mẫu biến môi trường (PORT, SSL, TURN).
- `report/`: `main.tex` và file PDF báo cáo.
- `report.md`: bản mô tả nhanh bằng Markdown.

2.2 Thư viện và cấu hình

Các thư viện chính theo `package.json`:

- `express`: phục vụ static file.
- `ws`: WebSocket signaling.
- `https`: chạy server HTTPS.
- `dotenv`: đọc biến môi trường.

Server dùng PORT từ môi trường (mặc định 3000). File `.env.example` gợi ý thêm các biến cho SSL/TURN, tuy nhiên hiện tại `server.js` vẫn đọc cert trực tiếp từ thư mục `certs/`.

Luồng tổng quát:

1. Client kết nối WebSocket đến signaling server.
2. Client gửi register rồi createRoom hoặc joinRoom theo nút người dùng chọn.
3. Server trả roomJoined và broadcast roomMembers cho các client trong phòng.
4. Một người bấm Bắt đầu gọi, load ICE servers, lấy local stream và tạo RTCPeerConnection tới từng peer khác.
5. Các peer trao đổi SDP/ICE, cập nhật trạng thái kết nối và log theo thời gian thực.

3 Thành phần server (signaling)

3.1 Cấu hình

- **HTTPS:** đọc certs/key.pem, certs/cert.pem. Nếu không có cert, server dừng.
- **Static:** express.static(..../public) để phục vụ client.
- **WebSocket:** ws để nhận/gửi signaling.

3.2 Quản lý phòng

Server lưu trong bộ nhớ:

```
rooms = Map<roomId, Map<name, WebSocket>>
```

- **joinRoom/createRoom:** thêm thành viên vào phòng, broadcast roomMembers.
- **leaveRoom/close WS:** xóa thành viên, broadcast memberLeft và danh sách mới nếu phòng còn người.
- **Phòng rỗng:** nếu phòng không còn ai, server xóa room khỏi bộ nhớ.
- **offer/answer/candidate:** chuyển tiếp 1-1 từ sender đến target.

3.3 Xử lý thông điệp signaling

Server chỉ đóng vai trò chuyển tiếp, không xử lý media. Luồng xử lý chính:

- register: lưu nickname hiện tại cho socket.
- createRoom: nếu đang ở phòng khác thì leaveRoom. Nếu phòng đã tồn tại thì trả error; nếu hợp lệ, tạo phòng, gửi roomJoined và broadcast roomMembers.
- joinRoom: nếu đang ở phòng khác thì leaveRoom. Nếu phòng chưa tồn tại hoặc tên đã dùng thì trả error; nếu hợp lệ, gửi roomJoined và broadcast roomMembers.
- offer/answer/candidate: gửi thẳng đến target tương ứng.
- endCall: broadcast tới mọi người trong phòng (trừ sender).
- leaveRoom hoặc close: loại bỏ thành viên, broadcast memberLeft (trường sender) và cập nhật roomMembers nếu phòng còn người.

4 Thành phần client

4.1 Giao diện

Giao diện gồm 2 màn:

- **Lobby:** nhập nickname, roomId, bấm Tạo phòng hoặc Vào phòng.
- **Room:** hiển thị thông tin phòng, trạng thái kết nối, nút Bắt đầu gọi/Dừng/Rời phòng, danh sách thành viên và grid video.

4.2 Cấu hình signaling URL

Client không hardcode IP, mà tự sinh URL theo protocol và host hiện tại:

```
const SIGNALING_URL = `${location.protocol === 'https:' ? 'wss:' : 'ws:'}//${location.host}`;
```

Điều này giúp chạy được cả HTTPS local và deploy cloud mà không phải sửa code.

4.3 Luồng gia nhập phòng

Hàm `enterRoom()` thực hiện:

1. Lấy nickname và roomId từ form.
2. Đặt cờ `isJoiningRoom` để tránh bấm nhiều lần.
3. Mở kết nối WebSocket đến signaling server.
4. Gửi `register` và `createRoom/joinRoom` theo nút người dùng.

Khi server trả `roomJoined`, client mới chuyển UI sang phòng. Khi nhận `roomMembers`, client cập nhật danh sách thành viên trên UI.

4.4 Xử lý lỗi

Nếu server trả `error` (phòng đã tồn tại, phòng chưa tồn tại, hoặc tên bị trùng), client hiển thị alert và đóng WebSocket nếu chưa vào phòng.

4.5 Trạng thái và cấu trúc dữ liệu

Client duy trì các biến trạng thái chính:

- `myName, myRoom`: thông tin người dùng/phòng hiện tại.
- `ws`: WebSocket đang kết nối.
- `localStream`: stream camera/mic.
- `isCallActive, isJoiningRoom`: cờ trạng thái cuộc gọi và tham gia phòng.
- `peerConnections`: `Map<peerName, RTCPeerConnection>` để quản lý các peer.
- `pendingCandidates`: hàng đợi candidate đến sớm khi chưa có remote description.

4.6 Xử lý WebSocket

Hàm `connectWS()` khởi tạo WebSocket và gắn các handler:

- `onopen`: gửi `register` và `createRoom/joinRoom` theo nút người dùng.
- `onmessage`: parse JSON, log hướng `send/recv` kèm `sender/target` và gọi `handleSignal()`.
- `onerror/onclose`: log trạng thái kết nối.

4.7 Ghi log và trạng thái UI

Client ghi log theo thời gian thực vào khu vực `log-container`: thời gian, loại sự kiện (`info/send/recv/error`) và nội dung message. Trạng thái tổng thể hiển thị ở `status-badge` dựa trên `connectionState` của các peer (`Connected/Connecting/Failed/Chờ`). Mỗi video có nhãn `conn-state`; sau khi kết nối thành công, nhãn này được cập nhật thành loại candidate (`relay/srflx/host`).

4.8 Tạo PeerConnection

Hàm `createPC(peerName, iceServers)` thiết lập WebRTC cho từng peer:

- Add toàn bộ audio/video tracks từ `localStream`.
- `ontrack`: tạo `MediaStream remote` và hiển thị vào grid.
- `onicecandidate`: gửi candidate về peer qua signaling.
- `onconnectionstatechange`: cập nhật `status-badge`, `conn-state` và gọi `logStats()` khi connected.
- `oniceconnectionstatechange`: log trạng thái ICE, cảnh báo khi failed.
- Cài timer 12 giây để log P2P failed, trying TURN... nếu chưa connected.

4.9 Gọi nhóm (mesh)

Khi một người nhấn `Bắt đầu gọi`:

1. Load danh sách ICE servers (STUN/TURN) bằng `loadIceServers()`.
2. Lấy local stream (camera/mic) bằng `getUserMedia`.
3. Đọc danh sách thành viên từ `members-list` trên UI, lọc bỏ chính mình.
4. Với mỗi peer: tạo `RTCPeerConnection`, add track, tạo offer và gửi offer.

Danh sách peer được lấy từ `roomMembers` do server broadcast, đảm bảo mỗi client tạo đủ $n-1$ kết nối trong room. Khi bắt đầu/ kết thúc cuộc gọi, client ghi log theo thời gian thực (timestamp) để phục vụ kiểm thử.

4.10 Nhận offer/answer/candidate

- **offer**: tạo hoặc reuse PC, `setRemoteDescription`, tạo answer, gửi answer.
- **answer**: `setRemoteDescription` và flush candidate đã xếp hàng; có nhánh dự phòng tạo PC mới nếu chưa tồn tại (trường hợp hiếm).
- **candidate**: `addIceCandidate` nếu đã có `remoteDescription`, ngược lại đưa vào hàng đợi.

4.11 Xử lý candidate đến sớm

Trong trường hợp candidate đến trước khi `remoteDescription` sẵn sàng, client đưa vào `pendingCandidates`. Sau khi `setRemoteDescription`, client gọi `flushPendingCandidates()` để add lại các candidate này.

4.12 Quản lý video

Mỗi peer được gắn một video trong `#video-grid`:

- Video local được mute tự động.
- Video remote hiển thị theo tên peer, kèm nhãn trạng thái kết nối ở góc phải.

Trạng thái kết nối của từng peer được hiển thị ngay trên khung video (`conn-state`) và badge tổng thể của phòng.

5 Signaling protocol

Type	Hướng	Payload chính
register	Client → Server	{type, name}
createRoom / joinRoom	Client → Server	{type, roomId, name}
roomMembers	Server → Client	{type, roomId, members[]}
roomJoined	Server → Client	{type, roomId, name}
error	Server → Client	{type, message}
offer	Client → Server → Client	{type, roomId, sender, target, offer}
answer	Client → Server → Client	{type, roomId, sender, target, answer}
candidate	Client → Server → Client	{type, roomId, sender, target, candidate}
leaveRoom	Client → Server	{type, roomId, sender}
memberLeft	Server → Client	{type, roomId, sender}
endCall	Client → Server → Client	{type, roomId, sender}

5.1 Luồng signaling chi tiết

Trình tự trao đổi SDP/ICE trong một cuộc gọi nhóm:

1. A kết nối WS, gửi `register` và `createRoom/joinRoom`; server trả `roomJoined` và broadcast `roomMembers`.
2. A bấm Bắt đầu gọi, load ICE servers, lấy local stream và tạo PC đến từng peer (B, C, ...).
3. A tạo offer, `setLocalDescription`, gửi offer tới từng peer.
4. B nhận offer, `setRemoteDescription`, tạo answer và gửi answer.
5. A nhận answer, `setRemoteDescription` để hoàn tất SDP.
6. Hai phía trao đổi candidate liên tục qua signaling; candidate đến sớm được xếp hàng.
7. Khi ICE chọn được đường, `connectionState` chuyển `connected`.
8. ontrack hiển thị remote stream trên grid.

6 ICE, STUN/TURN và fallback

6.1 Cấu hình ICE servers

Client có STUN mặc định và load TURN từ Metered:

```
const ICE_SERVERS = [
  { urls: 'stun:stun.l.google.com:19302' }
];

const res = await fetch('https://<app>.metered.live/api/v1/turn/credentials?
  apiKey=...');
```

Trong code hiện tại, URL Metered được hardcode trong `public/index.html`. Nếu gọi API thất bại, client fallback về `ICE_SERVERS` mặc định để vẫn có STUN hoạt động. Khi triển khai thật nên đưa key về server hoặc dùng biến môi trường.

Metered trả về danh sách TURN dạng:

- `turn:HOST:3478?transport=udp`
- `turn:HOST:3478?transport=tcp`
- `turns:HOST:5349?transport=tcp` (nếu có TLS)

6.2 Fallback và trạng thái kết nối

TURN được đặt trong `iceServers`, vì vậy trình duyệt tự động thương lượng ICE và có thể chọn relay khi P2P thất bại. Mỗi peer có timer 12 giây để cảnh báo P2P failed, trying TURN... nếu chưa connected. Trạng thái kết nối được hiển thị qua `connectionState` và `iceConnectionState` trên UI, đồng thời cập nhật `status-badge` và log ra console khi có lỗi (ví dụ `iceConnectionState = failed`).

UI có khu vực log để theo dõi luồng signaling (SEND/RECV), giúp kiểm tra nhanh tiến trình offer/answer/candidate trong lúc demo.

6.3 Thống kê ICE candidate

Sau khi kết nối connected, client gọi `getStats()` và tìm `candidate-pair` thành công để suy ra loại candidate (`host/srflx/relay`). Kết quả được gắn vào nhãn của từng peer, giúp chứng minh khi dùng TURN (relay) trong báo cáo và video demo.

7 Quản lý trạng thái và hangup

- **Hangup:** nếu phòng có nhiều người, UI hỏi xác nhận; sau đó gửi `endCall`, đóng toàn bộ peer connections, dừng local stream, reset UI và `status-badge`.
- **Leave room:** gọi `hangup` (không notify), gửi `leaveRoom`, đóng WS và quay về lobby.
- **Member left:** server broadcast `memberLeft`, client đóng PC tương ứng, xóa video và dọn candidate đã xếp hàng.
- Khi nhận `endCall` từ peer, client gọi `hangup` với `notifyPeers=false` để tránh vòng lặp broadcast.

8 Đối chiếu yêu cầu đề bài

8.1 TURN / Kết nối qua Internet

- **A1 - ICE servers:** client cấu hình STUN mặc định và fetch TURN từ Metered (UDP/TCP/TLS).
- **A2 - Fallback:** timer 12 giây và `iceConnectionState` báo lỗi, UI hiển thị trạng thái kết nối.
- **A3 - Thống kê:** log thời điểm bắt đầu/kết thúc, `connectionState`, `iceConnectionState`, và loại candidate qua `getStats()`.

8.2 Room & Group Call

- **B1 - Room:** UI có nút Tạo phòng/Vào phòng; server quản lý danh sách thành viên và broadcast `roomMembers`.
- **B2 - Mesh:** mỗi client tạo $n-1$ peer connections, hiển thị grid video, xử lý `memberLeft` để đóng PC.
- **B3 - Trạng thái:** `endCall/leaveRoom` dọn state và cho phép gọi lại.

9 Kết quả kiểm thử (bổ sung khi nộp)

Trong quá trình kiểm thử, cần ghi nhận:

- Trạng thái `connectionState` và `iceConnectionState` của từng peer.
- Loại candidate hiển thị trên UI (`host/srflx/relay`).
- Ảnh chụp grid video và log signaling (SEND/RECV) nếu cần.

9.1 Test cùng LAN (P2P)

- **Kết quả:** điền thêm sau khi test
- **Mình chứng:** đính kèm ảnh grid và log candidate type = `host/srflx`

9.2 Test khác mạng / 4G (TURN)

- **Kết quả:** điền thêm sau khi test
- **Mình chứng:** đính kèm log candidate type = `relay` hoặc trạng thái ICE failed

9.3 Test nhóm 3-4 người

- **Kết quả:** điền thêm sau khi test
- **Mình chứng:** đính kèm ảnh grid 3-4 video

10 Hạn chế và hướng phát triển

- Mesh topology tăng băng thông theo $O(n^2)$ khi tăng số lượng người.
- Chưa có SFU/MCU để tối ưu băng thông khi nhóm lớn.
- TURN credentials đang load trực tiếp trên client; có thể đưa về server để quản lý an toàn hơn.

- Nhánh xử lý `answer` khi chưa có PC tạo một kết nối tối giản, cần hoàn thiện để đồng nhất với `createPC`.
- Danh sách peer để gọi được lấy từ chuỗi hiển thị UI; có thể thay bằng state nội bộ để ổn định hơn.
- Room chỉ lưu trong bộ nhớ; khi server restart thì state bị mất.
- Chưa có cơ chế tự động reconnect WebSocket khi mất mạng.
- Cấu hình SSL từ biến môi trường chưa được dùng; server hiện đọc cert cố định trong `certs/`.

11 Hướng dẫn chạy nhanh

1. Cài dependencies: `npm install`.

2. Tạo cert HTTPS (self-signed):

```
openssl req -x509 -newkey rsa:2048 -keyout certs/key.pem -out certs/cert.
pem -days 365 -nodes -subj "/C=VN/ST=HCM/L=HCM/O=CSC11003/CN=
localhost"
```

3. Chạy server HTTPS:

```
node server/server.js
```

4. Truy cập `https://localhost:3000` và cho phép camera/mic.

5. (Tùy chọn) Deploy cloud (Heroku/Render) để test qua Internet theo README.

12 Kết luận

Đồ án triển khai hệ thống WebRTC gọi nhóm theo mesh, sử dụng signaling WebSocket và ICE (STUN/TURN) để hỗ trợ NAT/Firewall. Hệ thống hiển thị trạng thái kết nối rõ ràng, hỗ trợ join room, gọi nhóm nhiều người và quản lý hangup/leave ổn định. Hướng phát triển tiếp theo là chuyển sang kiến trúc SFU và cải thiện quản lý TURN credentials.